

Linear rotation: vectoring mode

$(y > 2x)$ and $(y > 4x)$

$(x \neq 1 \text{ and } y \neq 1)$

B1121123 巫煊崙

目錄

Introduction.....	2
Fixed point MATLAB.....	3
Block diagram.....	8
Fixed point Verilog.....	9
Synthesize Report.....	13
Code explanation.....	14
Floating point MATLAB.....	15
Floating point Verilog.....	20
Synthesize Report.....	26

Introduction

CORDIC 厲害的地方是不使用乘法器與除法器，單純透過簡單的移位與加減法計算複雜的數學式。

CORDIC 分成三種形式：

系統	m	用途
Circular	(m=1)	計算 sin、cos
Linear	(m=0)	乘法、除法
Hyperbolic	(m=-1)	雙曲函數、指數、平方根

我做的是 Linear CORDIC Vectoring Mode。透過調整 y ，使其逼近 0，在逼近的過程中，累加旋轉角度 z 。當 $y = 0$ 時， z 的累加結果就是 y 除以 x 的商。

迭代公式：

$$y_{i+1} = y_i - d_i x 2^{-i}$$

$$z_{i+1} = z_i + d_i 2^{-i}$$

d_i 由當下 y_i 的正負號決定：

- 若 $y_i \geq 0$ ，則 $d_i = 1$ （減法）
- 若 $y_i \leq 0$ ，則 $d_i = -1$ （加法）

我將輸入與輸出表示成 32-bit Q16.16 有號定點數，並進行 16 次 CORDIC 疊代，由於 Linear CORDIC 的商值收斂範圍為 $[-2, 2]$ ，當輸入商值超出此範圍時，需要先將被除數進行 pre-scale，讓商值在收斂範圍中，CORDIC 運算完成後，再根據 scale 將結果左移還原。

我讓 MATLAB 輸出兩個 txt 檔案：

data_input.txt，被除數與除數的 Q16.16 十六進位。

data_output.txt 是 CORDIC 所計算出的商值。

在 Verilog testbench 中，可以使用 \$readmemh 或 \$fscanf 讀取這些資料，並將 RTL 的輸出與 data_output.txt 進行比較。

Spec:

測試	5000 筆
除數大小	0.5 到 10.5
除數正負	隨機
商值大小	0.1 到 30.1
商值正負	隨機
定點格式	Q16.16

Fixed point MATLAB

```
clc; clear;
rng(42);
for k = 1:5000
    % 產生正負
    x_sign = randi([0, 1]) * 2 - 1;
    q_sign = randi([0, 1]) * 2 - 1;

    x_temp = (0.5 + rand() * 10) * x_sign;
    q_temp = (0.1 + rand() * 30) * q_sign;

    % Q16.16
    x_input(k) = floor(x_temp * 2^16) / 2^16;

    y_temp = x_input(k) * q_temp;
    y_input(k) = floor(y_temp * 2^16) / 2^16;
end

fid_in = fopen('data_input.txt', 'w');
fid_out = fopen('data_output.txt', 'w');

for idx = 1:5000
    y_val = y_input(idx);
    x_val = x_input(idx);
    exact_q = y_val / x_val;
```

```

% 模擬硬體輸入
x = abs(x_val) * 2^16;
y = abs(y_val) * 2^16;
z = 0;
shift_count = 0;

% PRE_SCALE 模擬 y_ext > x2_ext 與 y <= y >>> 1
while y >= x * 2
    y = floor(y / 2); % y >>> 1
    shift_count = shift_count + 1;
end

% CORDIC_OP
for iter = 0:15
    x_shifted = floor(x / 2^iter); % x >>> iter
    cordic_step = floor(2^16 / 2^iter); % LUT step
    if y >= 0
        y = y - x_shifted;
        z = z + cordic_step;
    else
        y = y + x_shifted;
        z = z - cordic_step;
    end
end

% POST_SCALE (z << shift_count) 補 sign
cordic = z * (2^shift_count);
if sign(y_val) ~= sign(x_val)
    cordic = -cordic;
end

cordic_q = cordic / 2^16;

abs_err = abs(cordic_q - exact_q);

exact_q_all(idx) = exact_q;
cordic_q_all(idx) = cordic_q;

```

```

if idx <= 3 || idx == 5000
    fprintf('組別 %04d\n', idx);
    fprintf('  輸入 Y          = %.8f\n', y_val);
    fprintf('  輸入 X          = %.8f\n', x_val);
    fprintf('  MATLAB 商值       = %.8f\n', exact_q);
    fprintf('  CORDIC 商值       = %.8f\n', cordic_q);
    fprintf('  誤差              = %.12e\n\n', abs_err);
elseif idx == 6
    fprintf('... 等 ...\n\n');
end

y_fixed = sfi(y_val, 32, 16);
x_fixed = sfi(x_val, 32, 16);

q_floor = floor(cordic_q * 2^16) / 2^16;
q_fixed = sfi(q_floor, 32, 16);

y_hex = hex(y_fixed);
x_hex = hex(x_fixed);
q_hex = hex(q_fixed);

fprintf(fid_in, '%s %s\n', y_hex, x_hex);
fprintf(fid_out, '%s\n', q_hex);
end

fclose(fid_in);
fclose(fid_out);

% 計算 MSE (dB) 、 SQNR (dB)
err_signal = exact_q_all - cordic_q_all;

mse_linear = mean(err_signal.^2);
mse_dB = 10 * log10(mse_linear);

signal_power = mean(exact_q_all.^2);
sqnr_dB = 10 * log10(signal_power / mse_linear);

```

```

fprintf('MSE & SQNR \n');
fprintf('測試 : 5000 筆\n');
fprintf('MSE (線性) : %e\n', mse_linear);
fprintf('MSE (dB) : %.4f dB\n', mse_dB);
fprintf('SQNR (dB) : %.4f dB\n', sqnr_dB);

```

Command Window

組別 0001	
輸入 Y	= -141.22631836
輸入 X	= -7.81994629
MATLAB 商值	= 18.05975555
CORDIC 商值	= 18.06005859
誤差	= 3.030482542989e-04
組別 0002	
輸入 Y	= 28.19404602
輸入 X	= -1.08084106
MATLAB 商值	= -26.08528390
CORDIC 商值	= -26.08544922
誤差	= 1.653155396788e-04
組別 0003	
輸入 Y	= 20.60864258
輸入 X	= 0.70584106
MATLAB 商值	= 29.19728479
CORDIC 商值	= 29.19775391
誤差	= 4.691122684193e-04
... 等 ...	
組別 5000	
輸入 Y	= 25.98788452
輸入 X	= -4.93321228
MATLAB 商值	= -5.26794369
CORDIC 商值	= -5.26794434
誤差	= 6.422507725645e-07
MSE & SQNR	
測試 : 5000 筆	
MSE (線性) :	6.175612e-08
MSE (dB) :	-72.0932 dB
SQNR (dB) :	96.9349 dB
>>	

SQNR 大約等於 $6.02 \times N + 1.76$ (dB)，其中 N 是有效小數位元數。我的 $N=16$ ，所以理論上的 SQNR 約為 $6 \times 16 = 96$ dB，但實際上是 96 dB，因為我在 MATLAB 使用了無條件捨去 `floor()` 來做移位，而不是四捨五入 `round()`，`floor()` 會有量化誤差。

除了這個原因，還有因為進行 16 次的移位加減，這 16 次的 `floor()` 會將微小的誤差不斷累積。

所以 SQNR 96 dB 是可以接受的範圍

txt 檔案裡有 5000 筆資料，以下只擷取部分

data_input.txt

data_output.txt

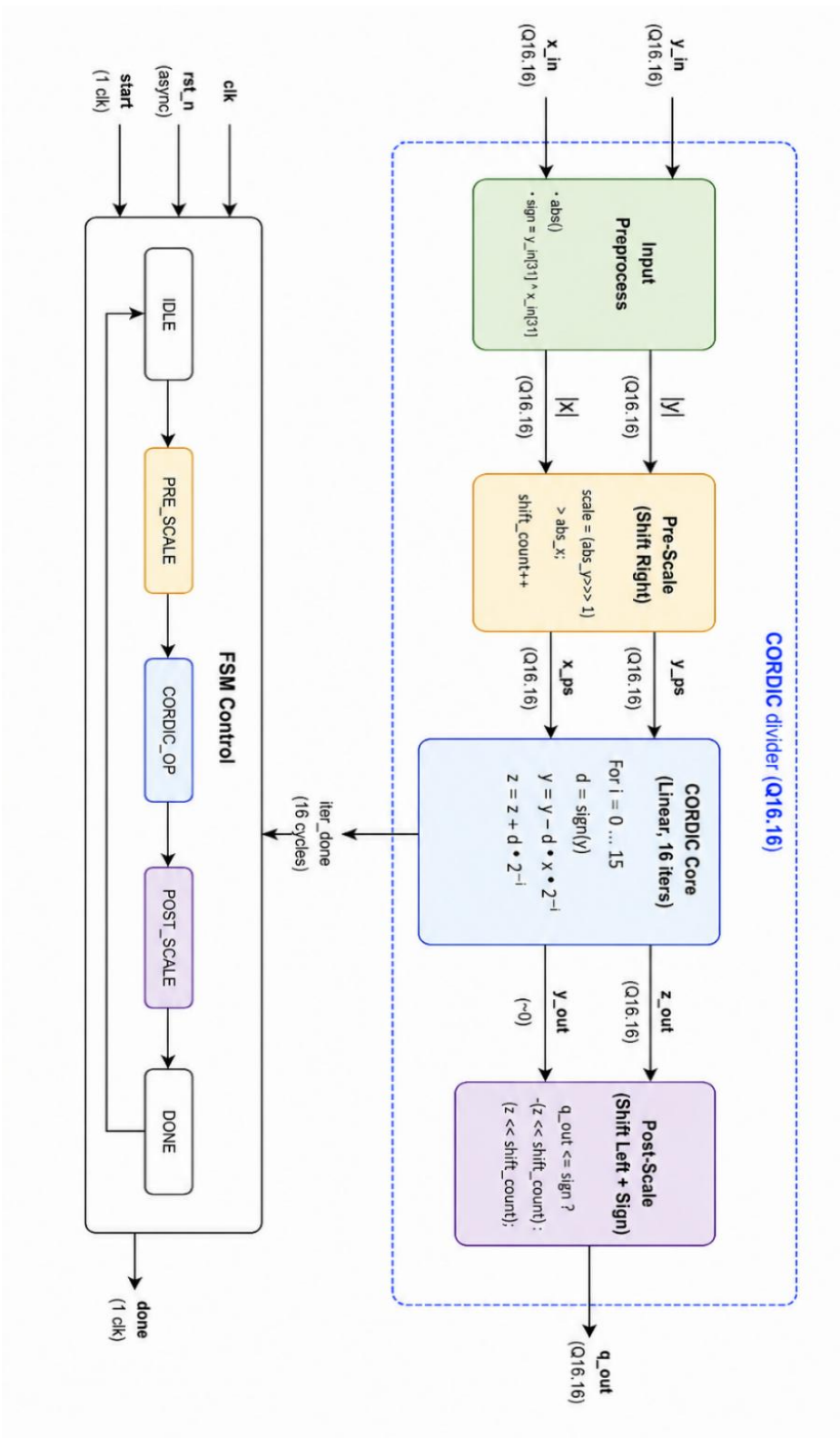
左邊是 Y(被除數) 右邊是 X(除數)

商值 Z

1	ff72c610	fff82e18
2	001c31ad	fffeeb4e
3	00149bd0	0000b4b2
4	fff3034e	00025178
5	ffd5693a	fffb2e38
6	ffda0daa	00036be3
7	ffd93b9e	fffd80d5
8	ffddb44e	00069350
9	ff08ac87	fff5d7fb
10	0061afc4	fff8a85d
11	00171af5	ffff27f7
12	ffc73437	fffc6205
13	ff11e2ff	000a3222
14	00b3d510	00067a9f
15	000963b3	ffff0c37
16	005eed35	fff7366e
17	ffd1dc70	fffe173b
18	ffce28e9	fff7c70d
19	ff59b76b	fff86e71
20	fff1649a	000415ac
21	0007a4c7	0003cf19
22	0095e48f	fff83435
23	ffdb8ab5	0001b229
24	007a701d	000835ad
25	fffd7bc6	0000c112
26	ffc80b95	fffc5b3f

1	00120f60
2	ffe5ea20
3	001d32a0
4	fffa65d8
5	0008d630
6	fff4e8b0
7	000f86f0
8	fffac8c8
9	00185a20
10	fff2b210
11	ffe49ea0
12	000fb3d0
13	ffe8a560
14	001bc1a0
15	fff623d0
16	fff53290
17	00182a60
18	00060fb8
19	0015f860
20	fffc6c84
21	000201bc
22	ffecc5e0
23	ffea80a0
24	000ee9f0
25	fffca9cc
26	000f5b70

Block diagram



Fixed point Verilog

```
module cordic_fixedpoint (
    input clk,
    input rst_n,
    input start,
    input signed [31:0] y_in, // Q16.16 被除數
    input signed [31:0] x_in, // Q16.16 除數
    output reg signed [31:0] q_out, // Q16.16 商
    output reg done
);

    localparam IDLE      = 3'd0;
    localparam PRE_SCALE = 3'd1;
    localparam CORDIC_OP = 3'd2;
    localparam POST_SCALE = 3'd3;
    localparam DONE      = 3'd4;

    reg signed [31:0] cordic_step;
    reg [2:0] curr_state, next_state;

    reg signed [31:0] x, y, z;
    reg [4:0] shift_count;
    reg [3:0] iter;

    // LUT (Q16.16)
    always @(*) begin
        case (iter)
            4'd0 : cordic_step = 32'h0001_0000; // 2^0 = 1.0
            4'd1 : cordic_step = 32'h0000_8000; // 2^-1 = 0.5
            4'd2 : cordic_step = 32'h0000_4000; // 2^-2 = 0.25
            4'd3 : cordic_step = 32'h0000_2000; // 2^-3 = 0.125
            4'd4 : cordic_step = 32'h0000_1000; // 2^-4 = 0.0625
            4'd5 : cordic_step = 32'h0000_0800; // 2^-5 = 0.03125
            4'd6 : cordic_step = 32'h0000_0400; // 2^-6 = 0.015625
            4'd7 : cordic_step = 32'h0000_0200; // 2^-7 = 0.0078125
            4'd8 : cordic_step = 32'h0000_0100; // 2^-8 = 0.00390625
            4'd9 : cordic_step = 32'h0000_0080; // 2^-9 = 0.001953125
            4'd10 : cordic_step = 32'h0000_0040; // 2^-10 = 0.0009765625
            4'd11 : cordic_step = 32'h0000_0020; // 2^-11 = 0.00048828125
            4'd12 : cordic_step = 32'h0000_0010; // 2^-12 = 0.000244140625
            4'd13 : cordic_step = 32'h0000_0008; // 2^-13 = 0.0001220703125
            4'd14 : cordic_step = 32'h0000_0004; // 2^-14 = 0.00006103515625
            4'd15 : cordic_step = 32'h0000_0002; // 2^-15 = 0.000030517578125
            default: cordic_step = 32'h0000_0000;
        endcase
    end

    wire signed [31:0] abs_y;
    wire signed [31:0] abs_x;
    wire sign;

    assign abs_y = (y[31]) ? -y : y;
    assign abs_x = (x[31]) ? -x : x;
    assign sign = y_in[31] ^ x_in[31];
```

```

wire scale;
assign scale = (abs_y >>> 1) > abs_x;

// fsm
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) curr_state <= IDLE;
    else curr_state <= next_state;
end

always @(*) begin
    case(curr_state)
        IDLE: next_state = start ? PRE_SCALE : IDLE;
        PRE_SCALE: next_state = (scale) ? PRE_SCALE : CORDIC_OP;
        CORDIC_OP: next_state = (iter == 4'd15) ? POST_SCALE : CORDIC_OP;
        POST_SCALE: next_state = DONE;
        DONE: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end

// iter
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        iter <= 4'd0;
    else if (curr_state == IDLE)
        iter <= 4'd0;
    else if (curr_state == CORDIC_OP)
        iter <= iter + 1'b1;
end

```

```

// shift_count
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        shift_count <= 5'd0;
    else if (curr_state == IDLE)
        shift_count <= 5'd0;
    else if (curr_state == PRE_SCALE && scale)
        shift_count <= shift_count + 1'b1;
end

// x
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        x <= 32'd0;
    else if (start)
        x <= x_in[31] ? -x_in : x_in; // |x|
end

// y
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        y <= 32'd0;
    else if (start)
        y <= y_in[31] ? -y_in : y_in; // |y|
    else if (scale)
        y <= y >>> 1;
    else if (curr_state == CORDIC_OP) begin
        if (y[31]) y <= y + (x >>> iter);
        else y <= y - (x >>> iter);
    end
end

```

```

// z
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        z <= 32'd0;
    else if (start)
        z <= 32'd0; // 初始商值 = 0
    else if(curr_state == CORDIC_OP && y[31])
        z <= z - cordic_step;
    else if(curr_state == CORDIC_OP && !y[31])
        z <= z + cordic_step;
end

// q_out
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q_out <= 32'd0;
    else if (curr_state == POST_SCALE)
        q_out <= sign ? ~(z << shift_count) : (z << shift_count);
end

// done
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        done <= 1'b0;
    else if (curr_state == POST_SCALE)
        done <= 1'b1;
    else
        done <= 1'b0;
end

endmodule

```

Simulation result

2. 163.25.97.100 (stu004)									
[PASS #4965] Y:	-216.866776	(ff27221b)	X:	-10.047546	(fff5f3d4)	=>	Got:	21.584473	(001595a0) Exp: 21.584473 (001595a0)
[PASS #4966] Y:	77.557648	(004d8ec2)	X:	2.598698	(00029738)	=>	Got:	29.937012	(001defe0) Exp: 29.937012 (001defe0)
[PASS #4967] Y:	67.728208	(0043b85f)	X:	4.110901	(00041c54)	=>	Got:	16.473145	(00107920) Exp: 16.473145 (00107920)
[PASS #4968] Y:	19.414832	(001369fe)	X:	7.549835	(00078c2)	=>	Got:	2.571472	(0002924c) Exp: 2.571472 (0002924c)
[PASS #4969] Y:	16.397629	(001065cb)	X:	-3.106308	(ffffc4c9)	=>	Got:	-5.278931	(ffffab898) Exp: -5.278931 (ffffab898)
[PASS #4970] Y:	10.810638	(000acfb6)	X:	-5.434418	(fffa90ca)	=>	Got:	-1.989288	(ffffe02be) Exp: -1.989288 (ffffe02be)
[PASS #4971] Y:	41.274384	(0029463e)	X:	7.800517	(0007cefb)	=>	Got:	5.285767	(00054928) Exp: 5.285767 (00054928)
[PASS #4972] Y:	232.616531	(00e89dd5)	X:	10.324677	(000a531e)	=>	Got:	22.529785	(001687a0) Exp: 22.529785 (001687a0)
[PASS #4973] Y:	1.141327	(0001242e)	X:	2.050934	(00020d8a)	=>	Got:	0.556488	(00008e76) Exp: 0.556488 (00008e76)
[PASS #4974] Y:	-37.123810	(ffdae04e)	X:	1.654327	(0001a782)	=>	Got:	-22.439941	(ffe98f60) Exp: -22.439941 (ffe98f60)
[PASS #4975] Y:	-92.815338	(ffa32f46)	X:	-9.674072	(ffff5370)	=>	Got:	9.593994	(00099810) Exp: 9.593994 (00099810)
[PASS #4976] Y:	3.549118	(00038c93)	X:	-1.804626	(ffff3204)	=>	Got:	-1.966705	(ffffe0886) Exp: -1.966705 (ffffe0886)
[PASS #4977] Y:	78.982758	(004efb96)	X:	-3.708664	(ffff4a95)	=>	Got:	-21.296387	(ffeab420) Exp: -21.296387 (ffeab420)
[PASS #4978] Y:	18.425125	(00126cd5)	X:	5.806366	(0005ce6e)	=>	Got:	3.173279	(00032c5c) Exp: 3.173279 (00032c5c)
[PASS #4979] Y:	-34.419601	(ffdd9495)	X:	1.356491	(00015b43)	=>	Got:	-25.374512	(ffe6a020) Exp: -25.374512 (ffe6a020)
[PASS #4980] Y:	250.303452	(00fa4daf)	X:	-9.214523	(ffff6c915)	=>	Got:	-27.163574	(ffe4d620) Exp: -27.163574 (ffe4d620)
[PASS #4981] Y:	139.168549	(008b2b26)	X:	-6.296295	(ffff9b426)	=>	Got:	-22.103027	(ffe9e5a0) Exp: -22.103027 (ffe9e5a0)
[PASS #4982] Y:	-79.852829	(ffb825ad)	X:	-10.028214	(ffff5f8c7)	=>	Got:	7.962769	(0007f678) Exp: 7.962769 (0007f678)
[PASS #4983] Y:	72.530838	(004887e5)	X:	3.191528	(00033108)	=>	Got:	22.726074	(0016b9e0) Exp: 22.726074 (0016b9e0)
[PASS #4984] Y:	-63.693634	(ffc04e6e)	X:	-8.474213	(fff7869a)	=>	Got:	7.516235	(00078428) Exp: 7.516235 (00078428)
[PASS #4985] Y:	20.964493	(0014f6e9)	X:	4.085785	(000415f6)	=>	Got:	5.130981	(00052188) Exp: 5.130981 (00052188)
[PASS #4986] Y:	-254.850449	(ff012649)	X:	-10.280991	(fff5b811)	=>	Got:	24.788574	(0018c9e0) Exp: 24.788574 (0018c9e0)
[PASS #4987] Y:	-16.450012	(ffef8ccc)	X:	-3.295288	(ffffcb468)	=>	Got:	4.992065	(0004fdf8) Exp: 4.992065 (0004fdf8)
[PASS #4988] Y:	164.053604	(00a40db9)	X:	-5.719315	(ffffa47db)	=>	Got:	-28.684082	(ffe350e0) Exp: -28.684082 (ffe350e0)
[PASS #4989] Y:	53.397781	(003565d5)	X:	1.780243	(0001c7be)	=>	Got:	29.995605	(001dfee0) Exp: 29.995605 (001dfee0)
[PASS #4990] Y:	70.405563	(004667d3)	X:	4.642990	(0004a49b)	=>	Got:	15.163818	(000f29f0) Exp: 15.163818 (000f29f0)
[PASS #4991] Y:	-201.061691	(ff3f6f35)	X:	7.394882	(00076517)	=>	Got:	-27.188965	(ffe4cfa0) Exp: -27.188965 (ffe4cfa0)
[PASS #4992] Y:	65.391174	(00416424)	X:	-7.882202	(fff81e28)	=>	Got:	-8.296143	(fff7b430) Exp: -8.296143 (fff7b430)
[PASS #4993] Y:	91.320053	(005b51ef)	X:	3.719101	(0003b817)	=>	Got:	24.554199	(00188de0) Exp: 24.554199 (00188de0)
[PASS #4994] Y:	212.735352	(00d4bc40)	X:	8.132935	(00082208)	=>	Got:	26.157715	(001a2860) Exp: 26.157715 (001a2860)
[PASS #4995] Y:	18.708618	(0012b568)	X:	-1.400085	(ffff9994)	=>	Got:	-13.362549	(fff2a330) Exp: -13.362549 (fff2a330)
[PASS #4996] Y:	-89.209534	(ffa6ca5c)	X:	8.997116	(0008ff43)	=>	Got:	-9.915283	(fff615b0) Exp: -9.915283 (fff615b0)
[PASS #4997] Y:	-113.211182	(ff8ec9f0)	X:	-4.758179	(ffffb3de8)	=>	Got:	23.793457	(0017cb20) Exp: 23.793457 (0017cb20)
[PASS #4998] Y:	-142.901825	(ff711922)	X:	-6.245789	(ffff9c114)	=>	Got:	22.879395	(0016e120) Exp: 22.879395 (0016e120)
[PASS #4999] Y:	251.180450	(00fb2e32)	X:	-9.510391	(ffff67d57)	=>	Got:	-26.411621	(ffe596a0) Exp: -26.411621 (ffe596a0)
[PASS #5000] Y:	25.987885	(0019fce6)	X:	-4.933212	(fffb1119)	=>	Got:	-5.267944	(fffbabb68) Exp: -5.267944 (fffbabb68)
測試完成: 共 5000 筆, PASS 5000, FAIL 0									
=====									
\$finish called from file "tb_fixedpoint.v", line 155.									
\$finish at simulation time 1250685000									
V C S S i m u l a t i o n R e p o r t									
Time: 1250685000 ps									
CPU Time: 0.300 seconds; Data structure size: 0.0Mb									
Wed Jun 17 09:16:37 2026									
CPU time: .157 seconds to compile + .178 seconds to elab + .112 seconds to link + .323 seconds in simulation									
[stu004@vlsi divider_fixedpoint]\$									

Synthesize Report

```

1
2 *****
3 Report : area
4 Design : cordic_fixedpoint
5 Version: U-2022.12-SP6
6 Date   : Wed Jun 17 11:07:07 2026
7 *****
8
9 Library(s) Used:
10
11   slow (File: /cad/CBDK/CBDK_IC_Contest_v2.5/SynopsysDC/db/slow.db)
12
13 Number of ports:          1051
14 Number of nets:          2876
15 Number of cells:         1900
16 Number of combinational cells: 1729
17 Number of sequential cells:  140
18 Number of macros/black boxes:  0
19 Number of buf/inv:        483
20 Number of references:     118
21
22 Combinational area:      21845.537869
23 Buf/Inv area:           2785.433407
24 Noncombinational area:   4924.157269
25 Macro/Black Box area:    0.000000
26 Net Interconnect area:   211681.061951
27
28 Total cell area:         26769.695137
29 Total area:              238450.757088
30 1
31

```

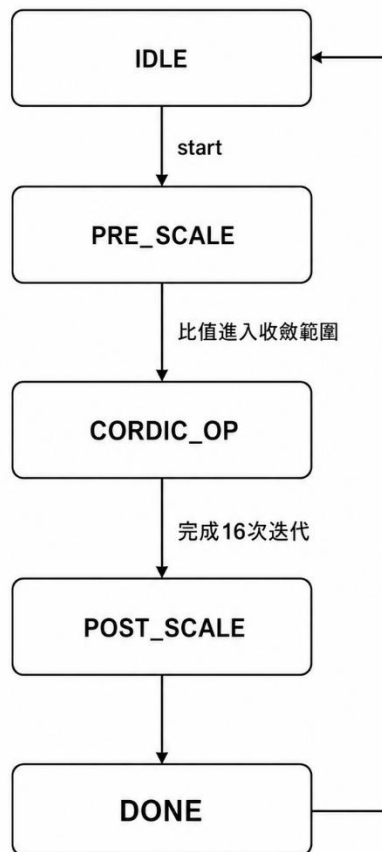
```

CORDIC_timing.L...
31 U555/Y (CLKINVX12)          0.06      0.80 r
32 U546/Y (CLKINVX8)           0.08      0.89 f
33 U527/Y (BUF2X0)             0.14      1.03 f
34 U558/Y (INVX4)              0.13      1.16 r
35 U745/Y (NAND2X2)            0.23      1.39 f
36 U712/Y (NOR2X4)            0.19      1.58 r
37 sub_126/B[2] (cordic_fixedpoint_DW01_sub_1) 0.00      1.58 r
38 sub_126/U14/Y (INVX6)       0.07      1.65 f
39 sub_126/U2_2/CO (ADDFHX1)    0.35      2.00 f
40 sub_126/U2_3/CO (ADDFX2)    0.37      2.37 f
41 sub_126/U2_4/CO (ADDFHX2)    0.26      2.63 f
42 sub_126/U2_5/CO (ADDFHX4)    0.21      2.84 f
43 sub_126/U2_6/CO (ADDFHX2)    0.24      3.08 f
44 sub_126/U2_7/CO (ADDFHX4)    0.21      3.29 f
45 sub_126/U2_8/CO (ADDFHX4)    0.19      3.48 f
46 sub_126/U2_9/CO (ADDFHX1)    0.30      3.79 f
47 sub_126/U2_10/CO (ADDFHX2)   0.26      4.05 f
48 sub_126/U2_11/CO (ADDFHX4)   0.21      4.26 f
49 sub_126/U2_12/CO (ADDFHX4)   0.20      4.47 f
50 sub_126/U2_13/CO (ADDFHX4)   0.20      4.67 f
51 sub_126/U2_14/CO (ADDFHX4)   0.20      4.87 f
52 sub_126/U2_15/CO (ADDFHX2)   0.24      5.10 f
53 sub_126/U2_16/CO (ADDFHX4)   0.23      5.34 f
54 sub_126/U2/Y (OR2X8)         0.17      5.51 f
55 sub_126/U6/Y (OR2X4)         0.19      5.70 f
56 sub_126/U43/Y (OR2X1)        0.37      6.07 f
57 sub_126/U7/Y (OR2X2)         0.31      6.38 f
58 sub_126/U42/Y (OR2X1)        0.38      6.76 f
59 sub_126/U8/Y (OR2X2)         0.31      7.06 f
60 sub_126/U41/Y (OR2X1)        0.38      7.44 f
61 sub_126/U9/Y (OR2X2)         0.31      7.75 f
62 sub_126/U40/Y (OR2X1)        0.38      8.13 f
63 sub_126/U10/Y (OR2X2)        0.31      8.44 f
64 sub_126/U39/Y (OR2X1)        0.38      8.81 f
65 sub_126/U11/Y (OR2X2)        0.31      9.13 f
66 sub_126/U15/Y (OR2X4)        0.22      9.34 f
67 sub_126/U24/Y (XNOR2XL)      0.33      9.68 f
68 sub_126/DIFF[30] (cordic_fixedpoint_DW01_sub_1) 0.00      9.68 f
69 U610/Y (AND2X4)              0.26      9.94 f
70 U665/Y (OR3X2)               0.25     10.18 f
71 z_reg[30]/D (DFFRX1)         0.00     10.18 f
72 data arrival time             10.18
73
74 clock clk (rise edge)         10.00     10.00
75 clock network delay (ideal)   0.50     10.50
76 clock uncertainty             -0.10     10.40
77 z_reg[30]/CK (DFFRX1)         0.00     10.40 r
78 library setup time           -0.22     10.18
79 data required time            10.18
80
81 data required time             10.18
82 data arrival time             -10.18
83
84 slack (MET)                   0.00
85

```

Code explanation

分成五個階段：



IDLE 等待開始

PRE_SCALE 判斷被除數是否太大，若超出收斂範圍，就 scale n 次，運算完成後，再乘上 2^{-n} ，才能變成商值。由於商值最大為 30.1，因此在目前測試內，最大縮放次數為 4 次。

CORDIC_OP 執行 16 次迭代；POST_SCALE 將縮放倍率與正負號補回

DONE 則表示運算完成。

Floating point MATLAB

```
clc;
clear;
rng(42);

FRAC_MASK = uint32(hex2dec('007FFFFF'));
HIDDEN_BIT= uint32(hex2dec('00800000'));
TWO       = int32(hex2dec('01000000')); % Q1.23
ONE       = int32(hex2dec('00800000')); % Q1.23

for k = 1:5000
    x_sign = single(2 * randi([0, 1]) - 1);
    q_sign = single(2 * randi([0, 1]) - 1);

    x_temp = single(0.5) + single(10.0) * rand(1, 1, 'single');
    q_temp = single(0.1) + single(30.0) * rand(1, 1, 'single');

    x_temp = single(x_sign * x_temp);
    q_temp = single(q_sign * q_temp);
    y_temp = single(x_temp * q_temp);

    x_input(k) = x_temp;
    y_input(k) = y_temp;
end

fid_in  = fopen('data_input.txt', 'w');
fid_out = fopen('data_output.txt', 'w');

for idx = 1:5000
    y_val  = single(y_input(idx));
    x_val  = single(x_input(idx));
    exact_q = single(y_val / x_val);

    y_uint = typecast(y_val, 'uint32');
    x_uint = typecast(x_val, 'uint32');
```

```

% 拆解 IEEE 754
y_sign_bit = uint32(bitsliceget(y_uint, 32, 32));
x_sign_bit = uint32(bitsliceget(x_uint, 32, 32));
sign_out = bitxor(y_sign_bit, x_sign_bit);

y_exp = uint32(bitsliceget(y_uint, 31, 24));
x_exp = uint32(bitsliceget(x_uint, 31, 24));

y_frac = uint32(bitsliceget(y_uint, 23, 1));
x_frac = uint32(bitsliceget(x_uint, 23, 1));

% 加入 hidden bit
x      = int32(bitor(HIDDEN_BIT, x_frac));
y      = int32(bitor(HIDDEN_BIT, y_frac));
z      = int32(0);
exponent = int32(y_exp) - int32(x_exp) + int32(127);

% CORDIC iter
for iter = 0:15
    x_shift = bitshift(x, -iter);
    cordic_step = bitshift(int32(1), 23 - iter);

    if y < 0
        y = y + x_shift;
        z = z - cordic_step;
    else
        y = y - x_shift;
        z = z + cordic_step;
    end
end

% NORMALIZE
if z >= TWO
    mantissa = bitshift(z, -1);
    exponent = exponent + int32(1);
elseif z < ONE
    mantissa = bitshift(z, 1);

```



```

        exponent = exponent - int32(1);
else
    mantissa = z;
end

% PACK
e_uint = uint32(exponent);
m_uint = uint32(mantissa);

sign_field = bitshift(sign_out, 31);

if exponent <= int32(0)
    q_uint = sign_field;
else
    exponent_field = bitshift(uint32(bitsliceget(e_uint, 8, 1)), 23);

    fraction_field = uint32(bitsliceget(m_uint, 23, 1));

    q_uint = bitor(sign_field, bitor(exponent_field, fraction_field));
end

cordic_q = typecast(q_uint, 'single');
abs_err = abs(double(cordic_q) - double(exact_q));

exact_q_all(idx) = exact_q;
cordic_q_all(idx) = cordic_q;
exponent_all(idx) = exponent;
mantissa_all(idx) = mantissa;
q_uint_all(idx) = q_uint;

if idx <= 3 || idx == 5000
    fprintf('組別 %04d\n', idx);
    fprintf('  輸入 Y          = %.8f\n', y_val);
    fprintf('  輸入 X          = %.8f\n', x_val);
    fprintf('  MATLAB 商值      = %.8f\n', exact_q);
    fprintf('  CORDIC 商值      = %.8f\n', cordic_q);
    fprintf('  誤差            = %.12e\n\n', abs_err);
elseif idx == 6

```

```

        fprintf('... 要等 1 min ...\n\n');
    end

    fprintf(fid_in, '%08X %08X\n', y_uint, x_uint);
    fprintf(fid_out, '%08X\n', q_uint);
end

fclose(fid_in);
fclose(fid_out);

exact_q_double = double(exact_q_all);
cordic_q_double = double(cordic_q_all);
err_signal      = exact_q_double - cordic_q_double;

mse_linear      = mean(err_signal .^ 2);
mse_dB          = 10 * log10(mse_linear);
signal_power    = mean(exact_q_double .^ 2);
snr_dB          = 10 * log10(signal_power / mse_linear);
max_abs_err     = max(abs(err_signal));
mean_abs_err    = mean(abs(err_signal));

fprintf('結果\n');
fprintf('測試      : %d 筆\n', 5000);
fprintf('MSE (線性)   : %.12e\n', mse_linear);
fprintf('MSE (dB)      : %.4f dB\n', mse_dB);
fprintf('SNR          : %.4f dB\n', snr_dB);

```

```

Command Window

組別 0001
輸入 Y          = -141.22618103
輸入 X          = -7.81993914
MATLAB 商值     = 18.05975533
CORDIC 商值     = 18.05957031
誤差           = 1.850128173828e-04

組別 0002
輸入 Y          = 28.19391441
輸入 X          = -1.08083606
MATLAB 商值     = -26.08528328
CORDIC 商值     = -26.08544922
誤差           = 1.659393310547e-04

組別 0003
輸入 Y          = 20.60876465
輸入 X          = 0.70584494
MATLAB 商值     = 29.19729805
CORDIC 商值     = 29.19824219
誤差           = 9.441375732422e-04

... 要等 1 min ...

組別 5000
輸入 Y          = 25.98782158
輸入 X          = -4.93319988
MATLAB 商值     = -5.26794434
CORDIC 商值     = -5.26794434
誤差           = 0.000000000000e+00

結果
測試           : 5000 筆
MSE ( 線性 )   : 1.086126898593e-07
MSE ( dB )     : -69.6412 dB
SQNR           : 94.4829 dB
>>

```

計算方式：

$$x = (-1)^s M_x 2^{(exp_x - 127)}$$

$$y = (-1)^s M_y 2^{(exp_y - 127)}$$

$$\frac{y}{x} = \frac{M_y}{M_x}$$

$$\text{另外算 } Exp = exp_y - exp_x + 127$$

Floating point Verilog

```
`timescale 1ns / 1ps

module cordic_floatpoint (
    input          clk,
    input          rst_n,
    input          start,
    input [31:0] y_in, // IEEE 754 single, 被除數
    input [31:0] x_in, // IEEE 754 single, 除數
    output reg [31:0] q_out, // IEEE 754 single, 商
    output reg      done
);

    localparam IDLE      = 3'd0;
    localparam CORDIC_OP = 3'd1;
    localparam NORMALIZE = 3'd2;
    localparam PACK      = 3'd3;
    localparam DONE      = 3'd4;

    reg [2:0] curr_state;
    reg [2:0] next_state;

    reg signed [31:0] x;
    reg signed [31:0] y;
    reg signed [31:0] z;
    reg signed [31:0] mantissa;

    reg  sign;
    reg signed [10:0] exponent;
    reg [4:0] iter;
```

```

reg signed [31:0] cordic_step;

// LUT Q1.23
always @(*) begin
    case (iter)
        5'd0 : cordic_step = 32'h0080_0000; // 2^0   = 1.0
        5'd1 : cordic_step = 32'h0040_0000; // 2^-1  = 0.5
        5'd2 : cordic_step = 32'h0020_0000; // 2^-2  = 0.25
        5'd3 : cordic_step = 32'h0010_0000; // 2^-3  = 0.125
        5'd4 : cordic_step = 32'h0008_0000; // 2^-4
        5'd5 : cordic_step = 32'h0004_0000; // 2^-5
        5'd6 : cordic_step = 32'h0002_0000; // 2^-6
        5'd7 : cordic_step = 32'h0001_0000; // 2^-7
        5'd8 : cordic_step = 32'h0000_8000; // 2^-8
        5'd9 : cordic_step = 32'h0000_4000; // 2^-9
        5'd10: cordic_step = 32'h0000_2000; // 2^-10
        5'd11: cordic_step = 32'h0000_1000; // 2^-11
        5'd12: cordic_step = 32'h0000_0800; // 2^-12
        5'd13: cordic_step = 32'h0000_0400; // 2^-13
        5'd14: cordic_step = 32'h0000_0200; // 2^-14
        5'd15: cordic_step = 32'h0000_0100; // 2^-15
        5'd16: cordic_step = 32'h0000_0080; // 2^-16
        5'd17: cordic_step = 32'h0000_0040; // 2^-17
        5'd18: cordic_step = 32'h0000_0020; // 2^-18
        5'd19: cordic_step = 32'h0000_0010; // 2^-19
        5'd20: cordic_step = 32'h0000_0008; // 2^-20
        5'd21: cordic_step = 32'h0000_0004; // 2^-21
        5'd22: cordic_step = 32'h0000_0002; // 2^-22
        default: cordic_step = 32'h0000_0000;
    endcase
end

```

```

// fsm
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        curr_state <= IDLE;
    else
        curr_state <= next_state;
end

always @(*) begin
    case(curr_state)
        IDLE:      next_state = start ? CORDIC_OP : IDLE;
        CORDIC_OP: next_state = (iter == 5'd22) ? NORMALIZE : CORDIC_OP;
        NORMALIZE: next_state = PACK; // 將 CORDIC 結果 NORMALIZE 到 [1,2)
        PACK:      next_state = DONE;
        DONE:      next_state = IDLE;
        default:   next_state = IDLE;
    endcase
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        x <= 32'b0;
    else if ((curr_state == IDLE) && start)
        x <= {8'b0, 1'b1, x_in[22:0]}; // 1.xxx
end

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        y <= 32'b0;
    else if ((curr_state == IDLE) && start)
        y <= {8'b0, 1'b1, y_in[22:0]}; // 1.xxx
    else if (curr_state == CORDIC_OP) begin
        if (y[31])
            y <= y + (x >>> iter);
        else
            y <= y - (x >>> iter);
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        z <= 32'd0;
    else if ((curr_state == IDLE) && start)
        z <= 32'd0;
    else if (curr_state == CORDIC_OP) begin
        if (y[31])
            z <= z - cordic_step;
        else
            z <= z + cordic_step;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        iter <= 5'd0;
    else if ((curr_state == IDLE) && start)
        iter <= 5'd0;
    else if ((curr_state == CORDIC_OP))
        iter <= iter + 1'b1;
end

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        sign <= 1'b0;
    else if ((curr_state == IDLE) && start)
        sign <= y_in[31] ^ x_in[31];
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        exponent <= 11'b0;
    else if ((curr_state == IDLE) && start) // Exponent = Ey - Ex + 127
        exponent <= {3'b0, y_in[30:23]} - {3'b0, x_in[30:23]} + 11'd127;
    else if (curr_state == NORMALIZE) begin
        if (z >= 32'h0100_0000) // 2
            exponent <= exponent + 1'b1;
        else if (z < 32'h0080_0000) // 1
            exponent <= exponent - 1'b1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        mantissa <= 32'b0;
    else if (curr_state == NORMALIZE) begin
        if (z >= 32'h0100_0000) // 2
            mantissa <= z >> 1;
        else if (z < 32'h0080_0000) // 1
            mantissa <= z << 1;
        else
            mantissa <= z;
    end
end
end

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q_out <= 32'd0;
    else if (curr_state == PACK)
        q_out <= {sign, exponent[7:0], mantissa[22:0]};
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        done <= 1'b0;
    else if (curr_state == PACK)
        done <= 1'b1;
    else
        done <= 1'b0;
end

endmodule

```

[PASS #4967] Y:	67.720291 (428770ca) X:	4.110906 (40838c8a) => Got:	16.473324 (4183c95e) Exp:	16.473145 (4183c900)
[PASS #4968] Y:	19.414070 (419b5004) X:	7.549847 (40f19858) => Got:	2.571452 (402492ac) Exp:	2.571411 (40249200)
[PASS #4969] Y:	16.397556 (41832e32) X:	-3.106293 (c046cd81) => Got:	-5.278818 (c0a8ec14) Exp:	-5.279053 (c0a8ee00)
[PASS #4970] Y:	10.810649 (412cf86b) X:	-5.434417 (c0ade6be) => Got:	-1.989294 (bffeaf12c) Exp:	-1.989319 (bffeaf200)
[PASS #4971] Y:	41.274380 (422518f7) X:	7.808517 (40f9df60) => Got:	5.285814 (40a92564) Exp:	5.285889 (40a92600)
[PASS #4972] Y:	232.616806 (43689de7) X:	10.324688 (412531ec) => Got:	22.530155 (41b43dc2) Exp:	22.529785 (41b43d00)
[PASS #4973] Y:	1.141341 (3f921778) X:	2.050943 (400342a5) => Got:	0.556496 (3f0e7686) Exp:	0.556503 (3f0e7700)
[PASS #4974] Y:	-37.124008 (c2147efc) X:	1.654336 (3fd3c14c) => Got:	-22.440422 (c1b385fc) Exp:	-22.440430 (c1b38600)
[PASS #4975] Y:	-92.815308 (c2b9a170) X:	-9.674069 (c11ac8fd) => Got:	9.594236 (411981fe) Exp:	9.593994 (41198100)
[PASS #4976] Y:	3.549119 (406324c3) X:	-1.804624 (bfe6fde8) => Got:	-1.966681 (bffb3c34) Exp:	-1.966736 (bffb3e00)
[PASS #4977] Y:	78.982475 (429df707) X:	-3.708650 (c06d5a87) => Got:	-21.296837 (c1aa5fec) Exp:	-21.295898 (c1aa5e00)
[PASS #4978] Y:	18.425150 (419366b5) X:	5.806373 (40b9cdce) => Got:	3.173265 (404b16c4) Exp:	3.173218 (404b1600)
[PASS #4979] Y:	-34.419922 (c209ae00) X:	1.356504 (3fada1ec) => Got:	-25.374001 (c1cafd4f) Exp:	-25.374023 (c1cafe00)
[PASS #4980] Y:	250.303421 (437a4dad) X:	-9.214521 (c1136eae) => Got:	-27.164028 (c1d94fee) Exp:	-27.163574 (c1d94f00)
[PASS #4981] Y:	139.168335 (430b2b18) X:	-6.296285 (c0c97b2b) => Got:	-22.103249 (c1b0d374) Exp:	-22.102539 (c1b0d200)
[PASS #4982] Y:	-79.852722 (c29fb498) X:	-10.028200 (c1207382) => Got:	7.962820 (40fecf6c) Exp:	7.962646 (40fece00)
[PASS #4983] Y:	72.531036 (42910fe4) X:	3.191536 (404c4222) => Got:	22.726067 (41b5cefc) Exp:	22.725586 (41b5ce00)
[PASS #4984] Y:	-63.693531 (c27ec62d) X:	-8.474199 (c1079652) => Got:	7.516170 (40f08476) Exp:	7.516235 (40f08500)
[PASS #4985] Y:	20.964523 (41a7b758) X:	4.085789 (4082bec8) => Got:	5.131083 (40a431d6) Exp:	5.130981 (40a43100)
[PASS #4986] Y:	-254.850189 (c37ed9a6) X:	-10.280981 (c1247ee6) => Got:	24.788509 (41c64ede) Exp:	24.788574 (41c64f00)
[PASS #4987] Y:	-16.449951 (c1839980) X:	-3.295279 (c052e5d9) => Got:	4.991980 (409fbc4c) Exp:	4.991943 (409fbc00)
[PASS #4988] Y:	164.053177 (43240d9d) X:	-5.719300 (c0b70481) => Got:	-28.684135 (c1e5791c) Exp:	-28.684570 (c1e57a00)
[PASS #4989] Y:	53.397934 (4255977c) X:	1.780248 (3fe3df27) => Got:	29.994682 (41eff51c) Exp:	29.995117 (41eff600)
[PASS #4990] Y:	70.405785 (428ccfc3) X:	4.643004 (4094937d) => Got:	15.163845 (41729f1c) Exp:	15.163574 (41729e00)
[PASS #4991] Y:	-201.061951 (c3490fdc) X:	7.394892 (40eca2f4) => Got:	-27.189308 (c1d983b4) Exp:	-27.188477 (c1d98200)
[PASS #4992] Y:	65.391068 (4282c83a) X:	-7.882189 (c0fc3ae4) => Got:	-8.296055 (c104bca4) Exp:	-8.296387 (c104be00)
[PASS #4993] Y:	91.320229 (42b6a3f5) X:	3.719108 (406e05dc) => Got:	24.554329 (41c46f44) Exp:	24.553711 (41c46e00)
[PASS #4994] Y:	212.735382 (4354bc42) X:	8.132936 (41022081) => Got:	26.157269 (41d14216) Exp:	26.157715 (41d14300)
[PASS #4995] Y:	18.708549 (4195ab1c) X:	-1.400079 (bfb335cd) => Got:	-13.362492 (c155ccc4) Exp:	-13.362793 (c155ce00)
[PASS #4996] Y:	-89.209679 (c2b26b5b) X:	8.997130 (410ff43f) => Got:	-9.915350 (c11ea546) Exp:	-9.915283 (c11ea500)
[PASS #4997] Y:	-113.210999 (c2e26c08) X:	-4.758171 (c09842f0) => Got:	23.792965 (41be57fe) Exp:	23.792480 (41be5700)
[PASS #4998] Y:	-142.901688 (c30ee6d5) X:	-6.245782 (c0c7dd73) => Got:	22.879707 (41b709a4) Exp:	22.879883 (41b70a00)
[PASS #4999] Y:	251.180313 (437b2e29) X:	-9.510386 (c1182a8a) => Got:	-26.411160 (c1d34a0e) Exp:	-26.411621 (c1d34b00)
[PASS #5000] Y:	25.987822 (41cfe70f) X:	-4.933200 (c09ddcc6) => Got:	-5.267943 (c0a892fe) Exp:	-5.267944 (c0a89300)

測試完成：共 5000 筆，PASS 5000，FAIL 0

ALL TESTS PASSED

\$finish called from file "tb_floatingpoint.v", line 325.

\$finish at simulation time 1450070000

V C S Simulation Report

Time: 1450070000 ps

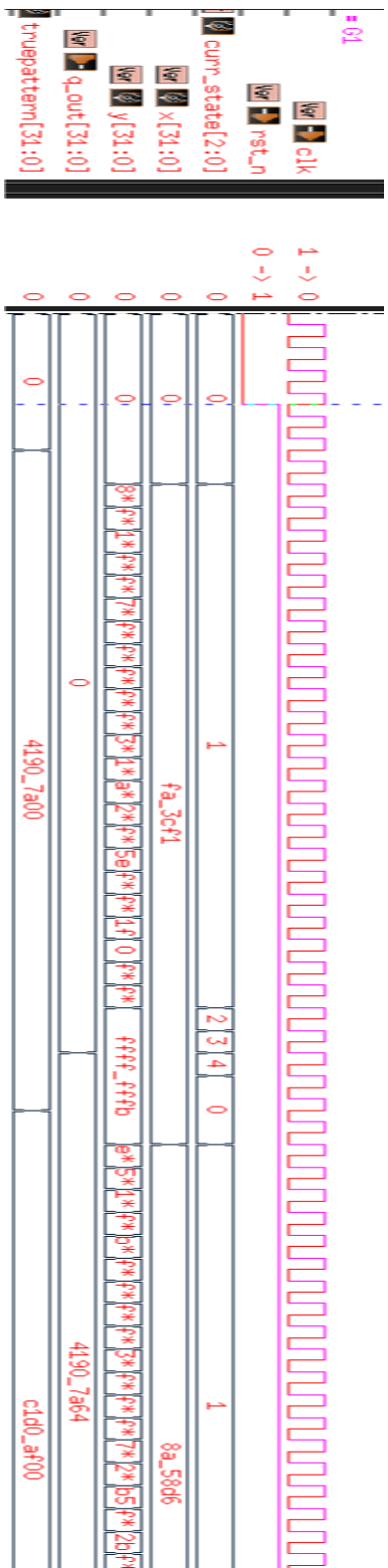
CPU Time: 0.390 seconds; Data structure size: 0.0Mb

Thu Jun 18 10:56:25 2026

CPU time: .165 seconds to compile + .207 seconds to elab + .146 seconds to link + .401 seconds in simulation

[2] - Done nWave

[stu004@vlsi divider_floatpoint]\$



Synthesize Report

sub_98/U2_7/C0 (ADDFHX4)	0.20	4.52 f
sub_98/U2_8/C0 (ADDFHX2)	0.22	4.74 f
sub_98/U2_9/C0 (ADDFHX2)	0.25	4.99 f
sub_98/U2_10/C0 (ADDFHX4)	0.21	5.20 f
sub_98/U2_11/C0 (ADDFHX4)	0.20	5.40 f
sub_98/U2_12/C0 (ADDFHX2)	0.22	5.62 f
sub_98/U2_13/C0 (ADDFHX2)	0.23	5.85 f
sub_98/U2_14/C0 (ADDFHX2)	0.23	6.08 f
sub_98/U2_15/C0 (ADDFHX1)	0.31	6.39 f
sub_98/U2_16/C0 (ADDFHX2)	0.26	6.66 f
sub_98/U2_17/C0 (ADDFHX4)	0.21	6.86 f
sub_98/U2_18/C0 (ADDFHX2)	0.22	7.09 f
sub_98/U2_19/C0 (ADDFHX2)	0.23	7.32 f
sub_98/U2_20/C0 (ADDFHX2)	0.25	7.57 f
sub_98/U2_21/C0 (ADDFHX4)	0.21	7.78 f
sub_98/U2_22/C0 (ADDFHX4)	0.23	8.01 f
sub_98/U12/Y (NAND2X1)	0.23	8.25 r
sub_98/U4/Y (NAND3X4)	0.17	8.42 f
sub_98/U3/Y (OR2X8)	0.19	8.61 f
sub_98/U2/Y (OR2X8)	0.17	8.78 f
sub_98/U1/Y (OR2X8)	0.17	8.94 f
sub_98/U28/Y (OR2X4)	0.20	9.14 f
sub_98/U29/Y (OR2X4)	0.21	9.35 f
sub_98/U30/Y (OR2X4)	0.21	9.56 f
sub_98/U17/Y (OR2X4)	0.20	9.75 f
sub_98/U15/Y (XNOR2X4)	0.17	9.93 r
sub_98/DIFF[31] (cordic_floatpoint_DW01_sub_1)	0.00	9.93 r
U509/Y (A0I22X4)	0.12	10.04 f
U517/Y (NAND2X4)	0.12	10.16 r
y_reg[31]/D (DFFRX1)	0.00	10.16 r
data arrival time		10.16
clock clk (rise edge)	10.00	10.00
clock network delay (ideal)	0.50	10.50
clock uncertainty	-0.10	10.40
y_reg[31]/CK (DFFRX1)	0.00	10.40 r
library setup time	-0.23	10.17
data required time		10.17
data required time		10.17
data arrival time		-10.16
slack (MET)		0.00

1
dc_shell> █

```

*****
Report : area
Design : cordic_floatpoint
Version: U-2022.12-SP6
Date   : Thu Jun 18 11:03:23 2026
*****

Library(s) Used:

    slow (File: /cad/CBDK/CBDK_IC_Contest_v2.5/SynopsysDC/db/slow.db)

Number of ports:          543
Number of nets:           1774
Number of cells:          1080
Number of combinational cells: 894
Number of sequential cells: 163
Number of macros/black boxes: 0
Number of buf/inv:        184
Number of references:      91

Combinational area:       13406.065177
Buf/Inv area:             1430.908219
Noncombinational area:    5338.322826
Macro/Black Box area:     0.000000
Net Interconnect area:    132767.338135

Total cell area:          18744.388003
Total area:               151511.726138
1
dc_shell> █

```